Users, Groups and Permissions Worksheet

CS 407 Intro To Linux

April 20, 2026

1 Introduction

Linux is a multiuser operating system.

Users can be assigned to groups.

You use groups to grant permissions to groups of users. Sometimes it makes sense to do this, like if you want to give every student on the computer access to a certain file.

Files and directories on you Linux computers have a user owner, a group owner, and permissions.

2 Note

Up until now we've been using the machines logged in as "root". This is not common and not recommended. We had to do this for a while though while you learned the skills needed to use the shell to configure users and groups.

Now you're ready to take your Linux skills to the next level! How exciting.

3 What you need to know

You need to learn some commands and concepts, and that's what we'll do in this lab. Here's what you must know:

- what do 4, 2, and 1 mean for permissions?
- what is the user owner of a file?
- what is the group owner of a file?

- understand how to read permissions like rwxr–r–
- **adduser** - add a user
- **addgroup** - create a new group
- **groups** - list the groups for your user
- **chmod** - change permissions of a file
- **chown** - change owner of a file
- **tree -pug** - tree, but with extra info
- **ls -l** - ls, but with extra info
- **deluser –remove-all-files** - delete a user
- **su** - change your user (prefer to do **su -**, though)
- **su -** - preferable to plain **su**
- **sudo** - a group whose members can act like root, without being root
- **sudo** - a command that users in the sudo group can use to act like root
- **usermod -aG** - add a user to a group

All of these commands work closely together, you'll see it's not like you're learning a bunch of things but really just one new concept. Let's get started.

4 Permissions

When you think of permissions, think of these three numbers and their meanings:

- 4 - read
- 2 - write
- 1 - execute

These numbers are added to expression permissions. For example, to express that someone has read, write and execute permissions on a file, you would say $4 + 2 + 1 = 7$. To express that someone only has read permission for a file, you would say 4. To say that someone has read and execute, but NO write, you would say $4 + 1 = 5$. How would you express read and write, but NO execute permission? If you said $4 + 2 = 6$, you'd be right!!

How the numbers work will become second nature to you after you use them a little bit. That's the purpose of this lab!!

The next thing to know is that permissions fall into three groups:

- user owner
- group owner
- everyone else

So if you wanted to say that a file's user owner can read, write and execute it (rwx), and that the group owner can just read and execute(r-x) and that everyone else can't do anything with the file (—) you would express this as the 3 digit number 750. The 7 is user owner permission, the 5 is the group owner permission, and the 0 is the everyone else permission.

As I said - this will become second nature to you .

5 ls -l

We can use ls -l to see the user owner, group owner, and permissions of a file. Let's create a file **hello.txt** in the root user's home directory and look at the ownership and permissions.

Listing 1: Use ls -l to see user owner, group owner and permissions.

```
root@machine$ cd $HOME
root@machine$ touch hello.txt
root@machine$ ls -l
-rw-r--r-- 1 root root 0 Apr 20 21:24 hello.txt
```

The first column of output shows the permissions and whether it's a file or a directory. **-rw-r–r–** starts with **-**. This means that **hello.txt** is a file. If it would have said **d** it would have been a directory. The next 9 characters show us the permissions. Let's look at them in groups of 3. The first three are **rw-**. This means that the user owner has read and write, but no execute. The next three are **r–**; this means that the group owner has read, but no write or execute. The last 3 are the same **r–**, which means that anyone else on the computer can only read the file.

The user owner and group owner are in the 3rd and 4th columns in the output of **ls -l**. The user owner is **root** and the group owner, in the fourth column, is also **root**.

6 tree

If you run the tree command like **tree -pug** you will see not only the tree structure, but also the pug (permissions, user owner and group owner). Give it a shot!

Listing 2: tree -pug gives similar information to ls -l

```
root@machine$ cd $HOME
root@machine$ touch hello.txt
root@machine$ tree -pug
[drwx------ root root ] .
   [-rw-r--r-- root root ] hello.txt
```

7 Activity 01 - Add users

Get a Linux computer and grab a friend. You'll work together to configure this machine in an interesting way. First, let's create 4 users. We'll create two teachers and two students (you and your friend). And for fun, we'll make a fifth user called randomperson, who is just some random person who also happens to use our machine.

Listing 3: create users

```
root@machine$ adduser your_name
# give a password
# repeat the password
# then it will ask you a bunch of weird questions like your room number.
# just ignore these questions - just hit enter
# finally type yes or whatever it asks for to confirm.
root@machine$ adduser your_friends_name
# same
root@machine$ adduser melvyn
# create me! You're making a teacher account.
root@machine$ adduser your_other_favorite_teacher
# create an account for your other favorite teacher
# then create an account for the random person
```

Great work! Now have a look inside /**home**. You should see that home directories were created for all five users!

Listing 4: see that user directories were created

```
root@machine$ ls /home
abraham beatrice melvyn natalie randomperson
```

You are currently logged in as root. You can change your user to a different user with **su - other_user**. For example:

Listing 5: change user with su, then go back to root by typing exit. You can see who you're logged in as at any time by typing whoami

```
root@machine$ su - abraham
```

```
abraham@machine$ whoami
abraham
abraham@machine$ exit
root@machine$ whoami
root
root@machine$ su - natalie
natalie@machine$ whoami
natalie
natalie@machine$ exit
root@machine$
```

CHECKPOINT! Did you create 5 users?

8 Activity 02 - Add groups

Now lets add some groups. We add groups to logically group users together. Let's create a few groups: professors, students, csmajors, cybersecmajors.

Listing 6: Let's add some groups

```
root@machine$ addgroup professors
root@machine$ addgroup students
root@machine$ addgroup csmajors
root@machine$ addgroup cybersecmajors
```

Now we can inspect /**etc/group**

Listing 7: Let's verify that the groups were created

```
root@machine$ cat /etc/group
...[long file]...
professors:other:data:here
students:other:data:here
csmajors:other:data:here
cybersecmajors:other:data:here
```

You can ignore the extra data in each line, but you'll see the different groups that you created there at the end of the file. To my knowledge, it should be at the end of the file. If it's not, scroll through the file.

Now lets put our new users in groups!

Listing 8: Let's verify that the groups were created

```
root@machine$ groups melvyn
melvyn users
root@machine$ usermod -aG professors melvyn
root@machine$ usermod -aG students melvyn
root@machine$ usermod -aG cybersecmajors melvyn
root@machine$ usermod -aG csmajors melvyn
root@machine$ groups melvyn
csmajors cybersecmajors professors students melvyn users
```

You'll see that melvyn is now a member of all of the groups we created. You should put me in all groups so I have access to all the data that a student would have, regardless of the major.

Then let's set up your groups. If your name is abe, and you're a student and a cyber security major, then you can set yourself up like this:

Listing 9: Add yourself to the proper groups

```
root@machine$ groups abraham
abraham users
root@machine$ usermod -aG students abraham
root@machine$ usermod -aG cybersecmajors abraham
root@machine$ groups abraham
cybersecmajors abraham students users
```

Now set up the other professor account, and the other student account similarly. Then let's move on.

CHECKPOINT! Did you create the four groups and add people to the right groups? Make sure you don't add the random person to any group.

9 Activity 03 - Permissions

Now we'll log in as different users and give permissions as appropriate to certain files.

To get started we need to set up a directory. This won't make sense just yet, so don't worry about what we're doing here. We are setting up a directory to play in for our lab, but the 1777 permission is complicated. Directory permissions are a topic of the next lab.

Listing 10: Going to modify directory permissions here, don't worry about this yet. Today we are just learning file permissions

```
root@machine$ mkdir /labexercise
root@machine$ chmod 1777 /labexercise
```

Now let's learn what the permissions do.

- log in as your user
- go to our lab exercise diretory
- create a file
- set the user owner of the file to your user
- set the group owner of the file to the student group
- set the permissions to rw-r——
- try logging out of your user and try logging in as other users. Users who are in the student group should be able to read, but not write. randomperson should not be able to read or write.

Listing 11: Create a test file

```
root@machine$ su - abraham
abraham@machine$ cd /labexercise
abraham@machine$ echo "hello" > test.txt
abraham@machine$ chown abraham:students test.txt
abraham@machine$ chmod 640 test.txt
abraham@machine$ ls -l test.txt
# make sure the permissions are actually 640, and make sure that the ownership is
    abrahams:students
```

The file is ready! Let's test it out!

Listing 12: melvyn can read, but not write to the file

```
abraham@machine$ exit
```

```
root@machine$ su - melvyn
melvyn@machine$ cat test.txt
hello
melvyn@machine$ echo hi >> test.txt
# Permission Denied
# This is because the student group members dont have w permission!
melvyn@machine$ echo hi >> test.txt
melvyn@machine$ exit
root@machine$ su - randomperson
randomperson@machine$ cat test.txt
# Permission Denied
# randomperson is not the user owner, so the 'rw-' permission isnt granted
# random person is not a student, so the 'r--' permission isnt granted
# therefore, random person is an "anyone else", in terms of permissions.
# So he has '---' permission
# in other words, he cannot read, write or execute!
```

10 Activity 03.b - Permissions again

Try again and see if you understand the execute permission.

Login as your user and create a file like

Listing 13: sample executable file: /labexercise/helloworld.sh

```
#!/usr/bin/env bash

echo "this is an executable file!"
```

Then set the permissions

Listing 14: Create a test file

```
root@machine$ su - abraham
abraham@machine$ cd /labexercise
abraham@machine$ vim helloworld.sh
# create the file shown above
abraham@machine$ chown abraham:students helloworld.sh
abraham@machine$ chmod 750 helloworld.sh
abraham@machine$ ls -l helloworld.sh
# make sure the permissions are actually 750, and make sure that the ownership is
    abrahams:students
abraham@machine$ ./helloworld.sh
this is an executable file!
```

Now, try to login as other users and make sure that only abraham can rwx, any student group member can r-x, but the random other person can do nothing.

CHECKPOINT! Did you play with permissions and logging in as different users to make sure you understand?

11 Activity 04 - sudo

A very immportant group in Linux is the **sudo** group. Members of the sudo group can use the sudo program to do things that only the root user (super user) can do. **root** is like the sysadmin account, and you don't usually want all the people on your computer to have root access to the machine.

But you might put a few trusted users in the sudo group so they can escalate their privilege and act as root when they need to install something, or do some other privileged task.

First let's see if you can install software on the computer using your account.

Listing 15: not allowed

```
root@machine$ su - abraham
abraham@machine$ apt install typeracer
# error
abraham@machine$ sudo apt install typeracer
# error
abraham@machine$ exit
root@machine$
```

You can't install anything! Now, let's say the only person who should have super powers on the machine is the randomperson.

Listing 16: sudo

```
root@machine$ usermod -aG sudo randomperson
root@machine$ su - randomperson
randomperson@machine$ apt install typeracer
# error
randomperson@machine$ sudo apt install -y typeracer
# it installs! woo hoo!
```

There are memes about sudo that you will now understand! You can google for "sudo make me a sandwich xkcd" to see a comic as an example.

12 Coming Up Next

Now you've understood adding users, groups, and what file permissions do. In the next lab we'll talk about directory permissions.